

Universal Peripheral Initialization Guide

Prepared for Embedded Learning

June 9, 2026

Contents

1	Introduction	2
2	Universal Initialization Pattern	2
2.1	Step 1: Enable Clock	2
2.2	Step 2: Configure GPIO Pins	2
2.3	Step 3: Configure Peripheral Registers	2
2.4	Step 4: Enable Interrupts (Optional)	3
2.5	Step 5: Start the Peripheral	3
3	Peripheral Examples	4
3.1	GPIO Initialization	4
3.2	UART Initialization	4
3.3	SPI Initialization	4
3.4	I2C Initialization	5
3.5	Timer Initialization	5
3.6	ADC Initialization	5
3.7	PWM Initialization	6

1 Introduction

This document provides a universal method for initializing microcontroller peripherals. The same pattern applies to STM32, ESP32, PIC, AVR, NXP, and most modern MCUs.

Every peripheral follows the same five-step initialization sequence:

1. Enable the peripheral clock
2. Configure GPIO pins
3. Configure peripheral registers
4. Enable interrupts (optional)
5. Start the peripheral

This guide includes simple examples for GPIO, UART, SPI, I2C, Timers, ADC, and PWM.

2 Universal Initialization Pattern

2.1 Step 1: Enable Clock

Most peripherals are disabled by default. You must enable the clock in the MCU's clock control module.

2.2 Step 2: Configure GPIO Pins

Peripherals require specific pin modes:

- Alternate Function (UART, SPI, I2C, PWM)
- Analog (ADC)
- Input/Output (GPIO)

2.3 Step 3: Configure Peripheral Registers

This includes:

- Baud rate or frequency
- Mode selection
- Prescalers
- Polarity and phase
- Sampling time

2.4 Step 4: Enable Interrupts (Optional)

If using interrupt-driven operation, configure:

- Peripheral interrupt enable bits
- NVIC interrupt routing

2.5 Step 5: Start the Peripheral

Finally, enable the peripheral:

- UART: UE bit
- Timer: CEN bit
- ADC: ADSTART
- SPI: SPE bit

3 Peripheral Examples

3.1 GPIO Initialization

```
1 /* 1. Enable clock */
2 RCC->AHB2ENR |= RCC_AHB2ENR_GPIOAEN;
3
4 /* 2. Configure PA5 as output */
5 GPIOA->MODER &= ~(3 << (5 * 2));
6 GPIOA->MODER |= (1 << (5 * 2));
7
8 /* 3. No peripheral registers */
9
10 /* 4. No interrupts */
11
12 /* 5. Use pin */
13 GPIOA->ODR |= (1 << 5);
```

3.2 UART Initialization

```
1 /* 1. Enable clocks */
2 RCC->APB2ENR |= RCC_APB2ENR_USART1EN;
3 RCC->AHB2ENR |= RCC_AHB2ENR_GPIOAEN;
4
5 /* 2. Configure PA9 (TX), PA10 (RX) */
6 GPIOA->AFR[1] |= (7 << 4) | (7 << 8);
7 GPIOA->MODER |= (2 << 18) | (2 << 20);
8
9 /* 3. Configure UART */
10 USART1->BRR = 0x1D4C; // 115200 baud
11 USART1->CR1 = USART_CR1_TE | USART_CR1_RE;
12
13 /* 4. Enable RX interrupt */
14 USART1->CR1 |= USART_CR1_RXNEIE;
15
16 /* 5. Start UART */
17 USART1->CR1 |= USART_CR1_UE;
```

3.3 SPI Initialization

```
1 /* 1. Enable clocks */
2 RCC->APB2ENR |= RCC_APB2ENR_SPI1EN;
3 RCC->AHB2ENR |= RCC_AHB2ENR_GPIOAEN;
4
5 /* 2. Configure PA5=SCK, PA6=MISO, PA7=MOSI */
6 GPIOA->MODER |= (2 << 10) | (2 << 12) | (2 << 14);
7 GPIOA->AFR[0] |= (5 << 20) | (5 << 24) | (5 << 28);
8
```

```

9  /* 3. Configure SPI */
10 SPI1->CR1 = SPI_CR1_MSTR | SPI_CR1_BR_0;
11
12 /* 4. Optional interrupts */
13
14 /* 5. Enable SPI */
15 SPI1->CR1 |= SPI_CR1_SPE;

```

3.4 I2C Initialization

```

1  /* 1. Enable clocks */
2  RCC->APB1ENR |= RCC_APB1ENR_I2C1EN;
3  RCC->AHB2ENR |= RCC_AHB2ENR_GPIOBEN;
4
5  /* 2. Configure PB6=SCL, PB7=SDA */
6  GPIOB->MODER |= (2 << 12) | (2 << 14);
7  GPIOB->AFR[0] |= (4 << 24) | (4 << 28);
8
9  /* 3. Configure I2C timing */
10 I2C1->TIMINGR = 0x00303D5B;
11
12 /* 4. Optional interrupts */
13
14 /* 5. Enable I2C */
15 I2C1->CR1 |= I2C_CR1_PE;

```

3.5 Timer Initialization

```

1  /* 1. Enable clock */
2  RCC->APB1ENR |= RCC_APB1ENR_TIM2EN;
3
4  /* 2. No GPIO needed */
5
6  /* 3. Configure timer */
7  TIM2->PSC = 7999;
8  TIM2->ARR = 1000;
9
10 /* 4. Enable interrupt */
11 TIM2->DIER |= TIM_DIER_UIE;
12
13 /* 5. Start timer */
14 TIM2->CR1 |= TIM_CR1_CEN;

```

3.6 ADC Initialization

```

1  /* 1. Enable clocks */
2  RCC->AHB2ENR |= RCC_AHB2ENR_ADCEN;

```

```

3  RCC->AHB2ENR |= RCC_AHB2ENR_GPIOAEN;
4
5  /* 2. Configure PA0 as analog */
6  GPIOA->MODER |= (3 << 0);
7
8  /* 3. Configure ADC */
9  ADC1->SMPR1 |= ADC_SMPR1_SMP0_2;
10
11 /* 4. Optional interrupts */
12
13 /* 5. Start ADC */
14 ADC1->CR |= ADC_CR_ADSTART;

```

3.7 PWM Initialization

```

1  /* 1. Enable clock */
2  RCC->APB1ENR |= RCC_APB1ENR_TIM3EN;
3  RCC->AHB2ENR |= RCC_AHB2ENR_GPIOAEN;
4
5  /* 2. Configure PA6 as AF for TIM3_CH1 */
6  GPIOA->MODER |= (2 << 12);
7  GPIOA->AFR[0] |= (2 << 24);
8
9  /* 3. Configure PWM */
10 TIM3->PSC = 79;
11 TIM3->ARR = 999;
12 TIM3->CCR1 = 500;
13
14 /* 4. Enable PWM mode */
15 TIM3->CCMR1 |= (6 << 4);
16 TIM3->CCER |= TIM_CCER_CC1E;
17
18 /* 5. Start timer */
19 TIM3->CR1 |= TIM_CR1_CEN;

```